

Formative Assignment: Advanced Linear Algebra (PCA)

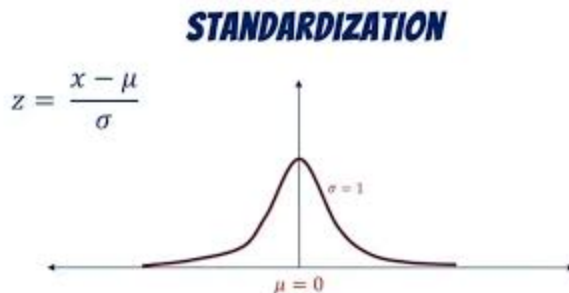
This notebook will guide you through the implementation of Principal Component Analysis (PCA). Fill in the missing code and provide the required answers in the appropriate sections. You will work with a dataset that is Africanized .

1. Make sure to display outputs for each code cell when submitting.
2. Do not write all your code on one cell
3. Do not use any libraries aside from numpy

Step 1: Load and Standardize the Data

Before applying PCA, we must standardize the dataset. Standardization ensures that all features have a mean of 0 and a standard deviation of 1, which is essential for PCA. Fill in the code to standardize the dataset.

STRICTLY - Write code that implements standardization based on the image below



Contributor: Milliam — Data loading, cleaning, missing-value handling, encoding, and standardization

Formative Assignment: Advanced Linear Algebra (PCA)

This notebook implements Principal Component Analysis (PCA) on the Life Expectancy dataset.

Assignment Requirements

1. Display outputs for each code cell when submitting.
2. Do not write all code in one cell.

3. NumPy is used for the PCA calculations.
4. The dataset is cleaned, standardized, and reduced using PCA.
5. The number of principal components is selected based on explained variance.

```
In [21]: # Step 1: Load, Clean and Standardize the Data

import numpy as np

# Load the dataset
raw = np.genfromtxt(
    "Life Expectancy Data.csv",
    delimiter=",",
    dtype=str,
    encoding="utf-8"
)

# Remove extra spaces
header = np.char.strip(raw[0])
body = np.char.strip(raw[1:])

# Display column names
print("Column names:")
print(header)

# Display first 5 rows
print("\nFirst 5 rows:")
print(body[:5])

# Identify non-numeric columns
country_index = np.where(header == "Country")[0][0]
status_index = np.where(header == "Status")[0][0]

print("\nNon-numeric columns:")
print(header[[country_index, status_index]])

# Encode Status:
# Developed = 1
# Developing = 0
status = np.where(
    body[:, status_index] == "Developed",
    1.0,
    0.0
)

# Select numerical columns
numeric_indices = np.array([
    i for i in range(len(header))
    if i != country_index and i != status_index
])

numeric_data = body[:, numeric_indices]

# Convert empty values to NaN
numeric_data[numeric_data == ""] = "nan"
```

```

# Convert numerical data to floats
numeric_data = numeric_data.astype(float)

# Count missing values
missing_values = np.isnan(numeric_data).sum(axis=0)

print("\nMissing values by numerical column:")

for name, count in zip(
    header[numeric_indices],
    missing_values
):
    print(name, ":", count)

# Replace missing values with column means
column_means = np.nanmean(
    numeric_data,
    axis=0
)

rows, columns = np.where(
    np.isnan(numeric_data)
)

numeric_data[rows, columns] = column_means[columns]

# Add encoded Status
clean_data = np.column_stack(
    (numeric_data, status)
)

print("\nCleaned data shape:")
print(clean_data.shape)

# Standardize the data
means = np.mean(
    clean_data,
    axis=0
)

stds = np.std(
    clean_data,
    axis=0
)

standardized_data = (
    clean_data - means
) / stds

print("\nFirst 5 rows of standardized data:")
print(standardized_data[:5])

print("\nMean after standardization:")
print(np.mean(standardized_data, axis=0))

```

```
print("\nStandard deviation after standardization:")  
print(np.std(standardized_data, axis=0))
```

Column names:

```
['Country' 'Year' 'Status' 'Life expectancy' 'Adult Mortality'
 'infant deaths' 'Alcohol' 'percentage expenditure' 'Hepatitis B'
 'Measles' 'BMI' 'under-five deaths' 'Polio' 'Total expenditure'
 'Diphtheria' 'HIV/AIDS' 'GDP' 'Population' 'thinness 1-19 years'
 'thinness 5-9 years' 'Income composition of resources' 'Schooling']
```

First 5 rows:

```
[[ 'Afghanistan' '2015' 'Developing' '65' '263' '62' '0.01' '71.27962362'
   '65' '1154' '19.1' '83' '6' '8.16' '65' '0.1' '584.25921' '33736494'
   '17.2' '17.3' '0.479' '10.1']
 [ 'Afghanistan' '2014' 'Developing' '59.9' '271' '64' '0.01'
   '73.52358168' '62' '492' '18.6' '86' '58' '8.18' '62' '0.1'
   '612.696514' '327582' '17.5' '17.5' '0.476' '10']
 [ 'Afghanistan' '2013' 'Developing' '59.9' '268' '66' '0.01'
   '73.21924272' '64' '430' '18.1' '89' '62' '8.13' '64' '0.1'
   '631.744976' '31731688' '17.7' '17.7' '0.47' '9.9']
 [ 'Afghanistan' '2012' 'Developing' '59.5' '272' '69' '0.01' '78.1842153'
   '67' '2787' '17.6' '93' '67' '8.52' '67' '0.1' '669.959' '3696958'
   '17.9' '18' '0.463' '9.8']
 [ 'Afghanistan' '2011' 'Developing' '59.2' '275' '71' '0.01'
   '7.097108703' '68' '3013' '17.2' '97' '68' '7.87' '68' '0.1'
   '63.537231' '2978599' '18.2' '18.2' '0.454' '9.5']]
```

Non-numeric columns:

```
['Country' 'Status']
```

Missing values by numerical column:

```
Year : 0
Life expectancy : 10
Adult Mortality : 10
infant deaths : 0
Alcohol : 194
percentage expenditure : 0
Hepatitis B : 553
Measles : 0
BMI : 34
under-five deaths : 0
Polio : 19
Total expenditure : 226
Diphtheria : 19
HIV/AIDS : 0
GDP : 448
Population : 652
thinness 1-19 years : 34
thinness 5-9 years : 34
Income composition of resources : 167
Schooling : 163
```

Cleaned data shape:

```
(2938, 21)
```

First 5 rows of standardized data:

```
[[ 1.6217623 -0.44444792 0.79158632 0.26882378 -1.1729584 -0.33557031
  -0.70586078 -0.11038437 -0.9647145 0.25535912 -3.27863785 0.92580631
  -0.73295174 -0.3234448 -0.52524766 0.38997511 2.81313018 2.77327898
```

```

-0.72540055 -0.57993072 -0.45939851]
[ 1.40498625 -0.98094995  0.85607167  0.28578638 -1.1729584 -0.33444132
-0.838704    -0.16812371 -0.98980951  0.27406024 -1.05148242  0.93414012
-0.8598765   -0.3234448  -0.52308258 -0.23093564  2.8814084   2.81790246
-0.74005007 -0.61056961 -0.45939851]
[ 1.1882102  -0.98094995  0.83188966  0.30274898 -1.1729584 -0.33459444
-0.75014185 -0.17353132 -1.01490451  0.29276135 -0.88016277  0.91330561
-0.77525999 -0.3234448  -0.52163233  0.35271542  2.92692722  2.86252595
-0.7693491  -0.6412085  -0.45939851]
[ 0.97143414 -1.02302854  0.86413234  0.32819289 -1.1729584 -0.33209644
-0.61729862  0.03204514 -1.03999951  0.31769617 -0.66601321  1.07581473
-0.64833523 -0.3234448  -0.5187229  -0.16831517  2.97244604  2.92946118
-0.80353132 -0.6718474  -0.45939851]
[ 0.75465809 -1.05458748  0.88831434  0.34515549 -1.1729584 -0.36786216
-0.57301755  0.05175675 -1.06007552  0.34263099 -0.62318329  0.8049662
-0.60602697 -0.3234448  -0.56489282 -0.181666   3.04072426  2.97408466
-0.84747987 -0.76376407 -0.45939851]]

```

Mean after standardization:

```

[ 2.37658769e-14  8.05346259e-16 -1.49944349e-16 -1.93476579e-17
-3.86953158e-17  5.07876019e-17  3.26491727e-17 -7.25537171e-18
 2.68448753e-16  4.83691447e-18 -2.47891867e-16  1.25759776e-16
 3.56722442e-16 -1.93476579e-17 -2.41845724e-17 -2.41845724e-17
 1.45107434e-16 -4.35322302e-17 -1.62036635e-16  5.36897506e-16
 2.66030296e-17]

```

Standard deviation after standardization:

```

[1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]

```

Step 3: Calculate the Covariance Matrix

The covariance matrix helps us understand how the features are related to each other. It is a key component in PCA.

In [22]: *# Step 3: Calculate the Covariance Matrix*

```

cov_matrix = np.cov(
    standardized_data,
    rowvar=False
)

print("Covariance Matrix:")
print(cov_matrix)

```

Covariance Matrix:

```
[[ 1.00034048  0.16968034 -0.07888762 -0.03742766 -0.04818399  0.03141067
   0.0894287  -0.08252106  0.10836359 -0.04295161  0.09385153  0.08188801
   0.13389894 -0.13978894  0.09338262  0.01495572 -0.04760834 -0.05064394
   0.23641364  0.20354047 -0.00186492]
 [ 0.16968034  1.00034048 -0.69659641 -0.19660192  0.39173167  0.38192117
   0.20384082 -0.15762747  0.55944572 -0.22257878  0.46173093  0.20805144
   0.47558026 -0.55664628  0.43063959 -0.01964439 -0.47232264 -0.46678809
   0.69271858  0.71530981  0.48212644]
 [-0.07888762 -0.69659641  1.00034048  0.07877394 -0.19047264 -0.2428962
  -0.1386381  0.03118466 -0.38157929  0.09416714 -0.27278642 -0.11091229
  -0.27310684  0.52390524 -0.27714725 -0.01250571  0.29996477  0.30547038
  -0.44021188 -0.4352566  -0.31527802]
 [-0.03742766 -0.19660192  0.07877394  1.00034048 -0.11385102 -0.08564137
  -0.17884426  0.50129897 -0.22729733  0.99696822 -0.17073187 -0.12660721
  -0.17521595  0.02523991 -0.10714551  0.54870843  0.46574868  0.47138839
  -0.1437117  -0.1918226  -0.1122898 ]
 [-0.04818399  0.39173167 -0.19047264 -0.11385102  1.00034048  0.33974993
   0.07547283 -0.05107237  0.3181786  -0.11081485  0.21381682  0.29499852
   0.21531523 -0.04866627  0.31869964 -0.03077513 -0.41708753 -0.40601887
   0.4162409  0.49771569  0.57956869]
 [ 0.03141067  0.38192117 -0.2428962  -0.08564137  0.33974993  1.00034048
   0.0116833  -0.05661495  0.22861504 -0.08788222  0.14725355  0.1734732
   0.14361866 -0.09789014  0.88844272 -0.02465661 -0.25127553 -0.25281091
   0.38050306  0.38823712  0.45441564]
 [ 0.0894287  0.20384082 -0.1386381  -0.17884426  0.07547283  0.0116833
   1.00034048 -0.09034769  0.13497445 -0.18447541  0.40865833  0.05010136
   0.5001279  -0.10244031  0.0623388  -0.10984827 -0.10517941 -0.10837113
   0.15104345  0.17181331  0.0956742 ]
 [-0.08252106 -0.15762747  0.03118466  0.50129897 -0.05107237 -0.05661495
  -0.09034769  1.00034048 -0.17598519  0.50798161 -0.13619263 -0.10460432
  -0.14190967  0.03090924 -0.06808277  0.23633032  0.22481869  0.22108241
  -0.11580349 -0.12265029 -0.07698127]
 [ 0.10836359  0.55944572 -0.38157929 -0.22729733  0.3181786  0.22861504
   0.13497445 -0.17598519  1.00034048 -0.23766678  0.28225199  0.23189333
   0.28115451 -0.24363051  0.27673894 -0.06325914 -0.5322059  -0.53909406
   0.48000075  0.50827848  0.31097842]
 [-0.04295161 -0.22257878  0.09416714  0.99696822 -0.11081485 -0.08788222
  -0.18447541  0.50798161 -0.23766678  1.00034048 -0.18876736 -0.12831306
  -0.19571717  0.03807447 -0.11067799  0.53604648  0.46778562  0.47225936
  -0.16158841 -0.20718194 -0.11523431]
 [ 0.09385153  0.46173093 -0.27278642 -0.17073187  0.21381682  0.14725355
   0.40865833 -0.13619263  0.28225199 -0.18876736  1.00034048  0.1301738
   0.67378265 -0.15954294  0.19404636 -0.03489368 -0.22001245 -0.22078525
   0.35551865  0.385963  0.2201734 ]
 [ 0.08188801  0.20805144 -0.11091229 -0.12660721  0.29499852  0.1734732
   0.05010136 -0.10460432  0.23189333 -0.12831306  0.1301738  1.00034048
   0.14564618 -0.00138319  0.12150845 -0.06672091 -0.26881525 -0.27533371
   0.14914549  0.21838446  0.29008378]
 [ 0.13389894  0.47558026 -0.27310684 -0.17521595  0.21531523  0.14361866
   0.5001279  -0.14190967  0.28115451 -0.19571717  0.67378265  0.14564618
   1.00034048 -0.16484294  0.1828568  -0.02546636 -0.22789743 -0.22118059
   0.37185571  0.39007657  0.21683728]
 [-0.13978894 -0.55664628  0.52390524  0.02523991 -0.04866627 -0.09789014
  -0.10244031  0.03090924 -0.24363051  0.03807447 -0.15954294 -0.00138319
  -0.16484294  1.00034048 -0.13455959 -0.02732773  0.20399156  0.20721008
```

```

-0.24753779 -0.21869419 -0.14864065]
[ 0.09338262 0.43063959 -0.27714725 -0.10714551 0.31869964 0.88844272
 0.0623388 -0.06808277 0.27673894 -0.11067799 0.19404636 0.12150845
 0.1828568 -0.13455959 1.00034048 -0.02562045 -0.26783579 -0.2724923
 0.440467 0.42963539 0.44606295]
[ 0.01495572 -0.01964439 -0.01250571 0.54870843 -0.03077513 -0.02465661
-0.10984827 0.23633032 -0.06325914 0.53604648 -0.03489368 -0.06672091
-0.02546636 -0.02732773 -0.02562045 1.00034048 0.23619779 0.23402055
-0.00795415 -0.02947494 -0.04110484]
[-0.04760834 -0.47232264 0.29996477 0.46574868 -0.41708753 -0.25127553
-0.10517941 0.22481869 -0.5322059 0.46778562 -0.22001245 -0.26881525
-0.22789743 0.20399156 -0.26783579 0.23619779 1.00034048 0.93942174
-0.40680019 -0.44629199 -0.36805925]
[-0.05064394 -0.46678809 0.30547038 0.47138839 -0.40601887 -0.25281091
-0.10837113 0.22108241 -0.53909406 0.47225936 -0.22078525 -0.27533371
-0.22118059 0.20721008 -0.2724923 0.23402055 0.93942174 1.00034048
-0.39591326 -0.4359256 -0.36642186]
[ 0.23641364 0.69271858 -0.44021188 -0.1437117 0.4162409 0.38050306
 0.15104345 -0.11580349 0.48000075 -0.16158841 0.35551865 0.14914549
 0.37185571 -0.24753779 0.440467 -0.00795415 -0.40680019 -0.39591326
 1.00034048 0.79647815 0.45745744]
[ 0.20354047 0.71530981 -0.4352566 -0.1918226 0.49771569 0.38823712
 0.17181331 -0.12265029 0.50827848 -0.20718194 0.385963 0.21838446
 0.39007657 -0.21869419 0.42963539 -0.02947494 -0.44629199 -0.4359256
 0.79647815 1.00034048 0.49161099]
[-0.00186492 0.48212644 -0.31527802 -0.1122898 0.57956869 0.45441564
 0.0956742 -0.07698127 0.31097842 -0.11523431 0.2201734 0.29008378
 0.21683728 -0.14864065 0.44606295 -0.04110484 -0.36805925 -0.36642186
 0.45745744 0.49161099 1.00034048]]

```

In a paragraph that has a maximum of 5 Lines, Explain why we need to compute a covariance matrix, provide atleast 2 reasons

The covariance matrix shows how the different features in the dataset vary in relation to each other. It is needed in PCA to identify relationships between features. The covariance matrix is also used for eigenvalue and eigenvector calculations. The eigenvectors determine the principal component directions, while the eigenvalues show the amount of variance captured by each direction.

Step 3: Calculate the Covariance Matrix

Explanation...

Contributor: Gasana — Covariance matrix implementation

[existing covariance matrix code]

In [39]: *# Task 3: Benchmark PCA Projection Performance*

```

import time

# Start timing
start_time = time.perf_counter()

```

```

# Vectorized PCA projection using NumPy
benchmark_reduced_data = np.dot(
    standardized_data,
    sorted_eigenvectors[:, :num_components]
)

# Stop timing
end_time = time.perf_counter()

execution_time = end_time - start_time

print("Performance Benchmark")
print("-----")
print("Original dataset shape:", standardized_data.shape)
print("Reduced dataset shape:", benchmark_reduced_data.shape)
print(f"PCA projection execution time: {execution_time:.8f} seconds")

```

Performance Benchmark

Original dataset shape: (2938, 21)

Reduced dataset shape: (2938, 15)

PCA projection execution time: 0.06874880 seconds

In [41]: *# Task 3: Benchmark PCA on the Larger Dataset*

```

start_time = time.perf_counter()

# Perform PCA projection on the larger dataset
large_reduced_data = np.dot(
    large_data,
    sorted_eigenvectors[:, :num_components]
)

end_time = time.perf_counter()

large_execution_time = end_time - start_time

print("Larger Dataset Performance Benchmark")
print("-----")
print("Larger dataset shape:", large_data.shape)
print("Reduced dataset shape:", large_reduced_data.shape)
print(f"PCA projection execution time: {large_execution_time:.8f} seconds")

```

Larger Dataset Performance Benchmark

Larger dataset shape: (29380, 21)

Reduced dataset shape: (29380, 15)

PCA projection execution time: 0.12698890 seconds

Task 3 Analysis

The PCA projection was implemented using NumPy vectorization with `np.dot()`, which avoids manually processing each row with Python loops.

For the original dataset of 2,938 rows and 21 features, the PCA projection took 0.06874880 seconds.

For the larger dataset of 29,380 rows and 21 features, the PCA projection took 0.12698890 seconds.

The larger dataset required more computation time because it contained ten times as many rows. However, the vectorized NumPy implementation was still able to process the larger dataset efficiently.

Contributor: Gasana — Performance optimization, larger-dataset testing, and benchmarking.

Step 4: Perform Eigendecomposition

Eigendecomposition of the covariance matrix will give us the eigenvalues and eigenvectors, which are essential for PCA. Fill in the code to compute the eigenvalues and eigenvectors of the covariance matrix.

Contributor: Gasana — Eigenvalue and eigenvector calculation

```
In [23]: # Step 4: Perform Eigendecomposition

eigenvalues, eigenvectors = np.linalg.eigh(
    cov_matrix
)

print("Eigenvalues:")
print(eigenvalues)

print("\nEigenvectors:")
print(eigenvectors)
```

Eigenvalues:

```
[2.70910516e-03 6.02811615e-02 1.03713297e-01 1.46736741e-01
1.97281117e-01 3.13508695e-01 3.73617592e-01 4.16869359e-01
4.22684047e-01 4.96245383e-01 5.63068019e-01 6.32127387e-01
7.64613881e-01 8.13465667e-01 8.55011746e-01 1.04498690e+00
1.26316118e+00 1.42109890e+00 1.85384847e+00 2.72715505e+00
6.53496644e+00]
```

Eigenvectors:

```
[[ -1.58537422e-03  3.05889767e-03 -3.60362915e-02  1.91460071e-02
  1.88261334e-02  1.55157030e-02  1.33198812e-01 -1.36849752e-01
 -1.83892554e-01 -9.25428148e-02 -1.47420523e-01  2.42278564e-01
 -2.19032631e-01  2.50032918e-01 -4.35219848e-02 -7.93430352e-01
 2.30158602e-01 -6.32562005e-02 -1.81237007e-01 -4.17799542e-02
 7.04362396e-02]
[ 2.69486661e-02 -2.34437557e-02 -1.86620833e-02  8.45610731e-01
 2.29860629e-01 -5.56323671e-02 -1.20787317e-02  3.31268938e-02
 2.42474784e-02  2.67966578e-02 -2.97271093e-02 -8.43641139e-02
 -1.15661624e-02 -1.43033662e-01 -5.18910551e-02  5.91062007e-02
 9.26567872e-02 -1.76854385e-01 -1.36135603e-01 -1.57888330e-01
 3.26702014e-01]
[ 3.77291986e-03 -1.36065373e-02 -5.71392051e-03  2.40434854e-01
 8.39259901e-02 -2.23475155e-02 -4.73040117e-01  5.31267851e-01
 5.66266997e-02 -2.32005967e-01 -6.70445963e-02  1.42390746e-01
 6.30717779e-02  9.25501166e-02  2.12770283e-01 -2.29459133e-01
 -1.59763303e-01  3.27642903e-01  1.50291385e-01  1.71485030e-01
 -2.26417928e-01]
[ -7.07359284e-01 -1.41535485e-03  4.31920181e-03  2.61235935e-02
 3.68359783e-02 -2.40870274e-03  2.20406349e-02 -1.22860388e-01
 2.71297802e-01 -3.09880128e-01  8.70420057e-02  1.80896292e-02
 4.64816365e-02  6.97292154e-02 -4.96451944e-03 -2.90610905e-02
 -1.47742153e-01 -2.84757374e-02 -1.00760747e-02 -4.90469408e-01
 -1.82407709e-01]
[ -9.30857682e-03 -8.72094617e-03  2.08623677e-02  2.14224634e-02
 -1.06904323e-01  1.76606834e-02  5.93601060e-01  3.26746357e-01
 -1.40367163e-01 -1.36528872e-01 -1.07249938e-01  3.30347353e-01
 -2.11860648e-01 -2.48341316e-01  3.37320595e-02  4.59193468e-02
 -3.46990991e-01  1.49335995e-01  2.39595131e-01 -9.72056078e-02
 2.23021923e-01]
[ 3.29639113e-04 -1.96029970e-02 -6.95291994e-01 -4.36557132e-02
 4.13692381e-02 -1.98475711e-02  6.85756599e-02  6.34285252e-02
 2.82710532e-02 -9.17922937e-03 -7.57552028e-03 -3.52957726e-02
 1.45202493e-01  2.20133941e-01 -8.52293853e-02  8.70763502e-02
 3.78489719e-01  2.93956414e-01  3.38839934e-01 -1.73453777e-01
 2.05690049e-01]
[ -2.68836027e-03  3.28508603e-03 -3.05155034e-02  5.99928243e-02
 9.39480129e-03  1.78103520e-01  1.59842480e-02  2.82339921e-02
 1.18537032e-01  1.85654412e-01  5.88499251e-01  3.69226825e-01
 -1.56564093e-01  2.85921287e-01 -6.42161441e-02  1.51253067e-01
 -7.51863547e-02  3.18748080e-01 -4.17470415e-01  5.31474769e-02
 1.17419889e-01]
[ -3.36021639e-03  7.08622959e-03 -8.11570369e-03  3.40720069e-02
 -1.84898440e-02 -2.19787469e-02 -9.23220142e-02  1.39752755e-01
 -3.08259320e-01  2.79574668e-01 -1.73216088e-01 -2.72971645e-01
 -5.18517749e-01  4.75773058e-01  1.12898701e-01  1.82778916e-01
 -1.84031461e-01 -5.35833918e-02  4.09561852e-02 -3.13378925e-01]
```

-1.18878257e-01]
 [-1.22782432e-03 1.39949671e-02 -3.17157182e-02 -7.30494592e-02
 -4.51295781e-02 -4.14707486e-02 -2.40776152e-02 1.06151743e-01
 -5.26593931e-01 -3.60543064e-01 5.02574682e-01 -2.79328807e-01
 2.53761137e-01 1.06118803e-01 1.53037594e-01 -8.62033441e-02
 -1.50528515e-01 -2.06254846e-01 -5.11470624e-03 -1.47654189e-02
 2.60780032e-01]
 [7.06049646e-01 -1.15362716e-02 1.09752784e-02 -8.14525867e-03
 3.39144863e-02 5.72632653e-03 3.17328549e-02 -1.23275722e-01
 2.68578604e-01 -3.18392611e-01 9.20382870e-02 2.67389078e-02
 3.27382668e-02 7.96858177e-02 -6.15592342e-03 -2.63914644e-02
 -1.51447401e-01 -2.57322587e-02 4.34890968e-03 -4.83644733e-01
 -1.88167954e-01]
 [2.28627185e-03 9.57774019e-03 -2.36926087e-02 -4.67022775e-02
 -4.92467415e-02 6.15331746e-01 -4.31658028e-02 -6.47093887e-02
 -1.03687541e-01 -2.25860548e-01 -4.28913985e-01 -1.99416939e-01
 1.55564584e-01 9.76391288e-03 3.20498754e-02 9.73045281e-02
 -1.05550321e-01 2.91269514e-01 -3.87975848e-01 -3.96367621e-02
 2.04173993e-01]
 [6.97025097e-04 8.39872251e-03 4.82283183e-02 -1.16030782e-02
 -5.03321190e-02 3.41988727e-02 -5.91639582e-02 1.22313294e-01
 1.33786002e-01 1.64019045e-01 5.83890292e-02 -3.06118175e-01
 6.20069087e-02 4.89508855e-02 -7.55139294e-01 -2.81498984e-01
 -3.63364972e-01 9.06545647e-02 1.36520644e-01 1.90557726e-02
 1.35700316e-01]
 [6.70798122e-03 -9.90579143e-03 2.21429082e-02 -1.29741371e-01
 -1.03169660e-02 -7.54999274e-01 1.50573517e-02 -6.22325263e-03
 1.72665696e-02 -7.00586149e-02 -2.30405641e-01 -6.54487007e-02
 1.12681024e-01 6.33805069e-02 6.64254024e-03 5.95576813e-02
 -1.12487097e-01 3.06779616e-01 -4.21395581e-01 -3.74368796e-02
 2.08598028e-01]
 [3.16979025e-03 -5.39025627e-03 -1.74625955e-03 2.50109751e-01
 4.96631431e-02 -1.95959734e-02 2.20906713e-01 -4.97069371e-01
 -3.92993778e-02 1.76051060e-01 1.30098707e-01 -2.83565698e-01
 4.67501193e-02 -2.69557041e-02 3.19681749e-01 -2.08905359e-01
 -2.45930881e-01 4.61969771e-01 2.14971080e-01 1.34303457e-01
 -1.47996943e-01]
 [-4.69017718e-03 2.84287864e-02 7.09194737e-01 3.23077398e-03
 -3.92175849e-02 2.38964936e-02 4.33033572e-02 5.39491776e-02
 -1.04834281e-03 -1.50773569e-02 2.37596733e-02 -2.34198652e-02
 1.34021259e-01 2.41031213e-01 -1.26615385e-02 5.83408704e-02
 4.15131855e-01 2.75506349e-01 2.82975956e-01 -1.75088765e-01
 2.22203984e-01]
 [9.32778810e-03 7.60963979e-03 -6.51503097e-03 -1.11847374e-02
 -2.00746351e-02 4.23088420e-02 -4.85778608e-02 8.69925918e-02
 -2.33206830e-01 5.48983973e-01 -7.80774284e-02 2.87997407e-01
 5.77182275e-01 -4.91285176e-02 9.58612844e-02 -1.23097722e-01
 -1.45398554e-01 -5.31839013e-02 -5.29263837e-02 -3.86607442e-01
 -8.14147814e-02]
 [-3.03431175e-03 -7.04408456e-01 2.27704022e-02 -1.39394921e-02
 -1.95625116e-02 -6.36857375e-05 2.22870627e-02 1.25940039e-01
 -2.37681000e-01 5.11856821e-04 1.35396750e-01 -1.44096328e-01
 -1.12415262e-01 -3.35539528e-01 -1.51682073e-01 -3.12912067e-02
 2.41640497e-01 2.21880775e-01 -1.55460332e-01 -1.71183225e-01
 -2.83645007e-01]
 [8.25201925e-03 7.07751519e-01 -2.48856094e-02 1.76812324e-02

```

-1.10341369e-02 -2.22289761e-02  2.71128668e-02  1.35496972e-01
-2.06713912e-01 -6.93025309e-03  1.27032685e-01 -1.39528455e-01
-1.20647774e-01 -3.52814562e-01 -1.37022230e-01 -3.27964548e-02
 2.36917694e-01  2.25104876e-01 -1.55418987e-01 -1.73513639e-01
-2.82920925e-01]
[ 5.08262186e-03 -9.30743184e-03 -6.00149654e-02 -7.86660817e-03
-6.90459267e-01  1.47866458e-02 -1.95725343e-01  8.53628127e-02
 3.05842823e-01  1.54206242e-01  1.00191717e-01 -1.62188771e-01
-1.56483310e-01 -2.18054055e-01  3.00074244e-01 -2.04178302e-01
 5.13184184e-02 -1.30087585e-02 -2.36314742e-02 -1.66361124e-01
 2.91903776e-01]
[-2.78100191e-03 -3.98796551e-03  3.41863502e-02 -3.58345532e-01
 6.59364171e-01  7.67824464e-02 -7.07448517e-02  1.49565187e-01
 2.04127562e-01  1.75786976e-01  8.71607380e-02 -1.68244849e-01
-1.65041755e-01 -2.33438738e-01  2.51701805e-01 -1.81668592e-01
-2.81525816e-02  1.75385839e-02 -4.59144039e-03 -1.42465756e-01
 3.07221438e-01]
[-2.51904924e-03  6.48368470e-03  4.86122621e-03 -5.23248197e-02
 1.34259731e-02 -4.81320398e-02 -5.35965324e-01 -4.29692292e-01
-3.00772588e-01 -8.93819646e-02 -1.44996788e-02  3.42230618e-01
-2.19080847e-01 -2.46762769e-01 -1.52818516e-01  9.04021137e-02
-1.20768217e-01  1.43093423e-01  2.30173446e-01 -1.40656475e-01
 2.42434444e-01]]

```

Step 5: Sort Principal Components

Sort the eigenvectors based on their corresponding eigenvalues in descending order. The higher the eigenvalue, the more important the eigenvector. Complete the code to sort the eigenvectors and print the sorted components.

How Is Explained Variance Used In PCA?

Contributor: Gasana — Principal component sorting

In [24]: *# Step 5: Sort Principal Components*

```

sorted_indices = np.argsort(
    eigenvalues
)[::-1]

sorted_eigenvalues = eigenvalues[
    sorted_indices
]

sorted_eigenvectors = eigenvectors[
    :, sorted_indices
]

print("Sorted eigenvalues:")
print(sorted_eigenvalues)

print("\nSorted eigenvectors:")
print(sorted_eigenvectors)

```

Sorted eigenvalues:

```
[6.53496644e+00 2.72715505e+00 1.85384847e+00 1.42109890e+00
 1.26316118e+00 1.04498690e+00 8.55011746e-01 8.13465667e-01
 7.64613881e-01 6.32127387e-01 5.63068019e-01 4.96245383e-01
 4.22684047e-01 4.16869359e-01 3.73617592e-01 3.13508695e-01
 1.97281117e-01 1.46736741e-01 1.03713297e-01 6.02811615e-02
 2.70910516e-03]
```

Sorted eigenvectors:

```
[[ 7.04362396e-02 -4.17799542e-02 -1.81237007e-01 -6.32562005e-02
 2.30158602e-01 -7.93430352e-01 -4.35219848e-02 2.50032918e-01
 -2.19032631e-01 2.42278564e-01 -1.47420523e-01 -9.25428148e-02
 -1.83892554e-01 -1.36849752e-01 1.33198812e-01 1.55157030e-02
 1.88261334e-02 1.91460071e-02 -3.60362915e-02 3.05889767e-03
 -1.58537422e-03]
 [ 3.26702014e-01 -1.57888330e-01 -1.36135603e-01 -1.76854385e-01
 9.26567872e-02 5.91062007e-02 -5.18910551e-02 -1.43033662e-01
 -1.15661624e-02 -8.43641139e-02 -2.97271093e-02 2.67966578e-02
 2.42474784e-02 3.31268938e-02 -1.20787317e-02 -5.56323671e-02
 2.29860629e-01 8.45610731e-01 -1.86620833e-02 -2.34437557e-02
 2.69486661e-02]
 [-2.26417928e-01 1.71485030e-01 1.50291385e-01 3.27642903e-01
 -1.59763303e-01 -2.29459133e-01 2.12770283e-01 9.25501166e-02
 6.30717779e-02 1.42390746e-01 -6.70445963e-02 -2.32005967e-01
 5.66266997e-02 5.31267851e-01 -4.73040117e-01 -2.23475155e-02
 8.39259901e-02 2.40434854e-01 -5.71392051e-03 -1.36065373e-02
 3.77291986e-03]
 [-1.82407709e-01 -4.90469408e-01 -1.00760747e-02 -2.84757374e-02
 -1.47742153e-01 -2.90610905e-02 -4.96451944e-03 6.97292154e-02
 4.64816365e-02 1.80896292e-02 8.70420057e-02 -3.09880128e-01
 2.71297802e-01 -1.22860388e-01 2.20406349e-02 -2.40870274e-03
 3.68359783e-02 2.61235935e-02 4.31920181e-03 -1.41535485e-03
 -7.07359284e-01]
 [ 2.23021923e-01 -9.72056078e-02 2.39595131e-01 1.49335995e-01
 -3.46990991e-01 4.59193468e-02 3.37320595e-02 -2.48341316e-01
 -2.11860648e-01 3.30347353e-01 -1.07249938e-01 -1.36528872e-01
 -1.40367163e-01 3.26746357e-01 5.93601060e-01 1.76606834e-02
 -1.06904323e-01 2.14224634e-02 2.08623677e-02 -8.72094617e-03
 -9.30857682e-03]
 [ 2.05690049e-01 -1.73453777e-01 3.38839934e-01 2.93956414e-01
 3.78489719e-01 8.70763502e-02 -8.52293853e-02 2.20133941e-01
 1.45202493e-01 -3.52957726e-02 -7.57552028e-03 -9.17922937e-03
 2.82710532e-02 6.34285252e-02 6.85756599e-02 -1.98475711e-02
 4.13692381e-02 -4.36557132e-02 -6.95291994e-01 -1.96029970e-02
 3.29639113e-04]
 [ 1.17419889e-01 5.31474769e-02 -4.17470415e-01 3.18748080e-01
 -7.51863547e-02 1.51253067e-01 -6.42161441e-02 2.85921287e-01
 -1.56564093e-01 3.69226825e-01 5.88499251e-01 1.85654412e-01
 1.18537032e-01 2.82339921e-02 1.59842480e-02 1.78103520e-01
 9.39480129e-03 5.99928243e-02 -3.05155034e-02 3.28508603e-03
 -2.68836027e-03]
 [-1.18878257e-01 -3.13378925e-01 4.09561852e-02 -5.35833918e-02
 -1.84031461e-01 1.82778916e-01 1.12898701e-01 4.75773058e-01
 -5.18517749e-01 -2.72971645e-01 -1.73216088e-01 2.79574668e-01
 -3.08259320e-01 1.39752755e-01 -9.23220142e-02 -2.19787469e-02
 -1.84898440e-02 3.40720069e-02 -8.11570369e-03 7.08622959e-03]
```

-3.36021639e-03]

[2.60780032e-01 -1.47654189e-02 -5.11470624e-03 -2.06254846e-01
-1.50528515e-01 -8.62033441e-02 1.53037594e-01 1.06118803e-01
2.53761137e-01 -2.79328807e-01 5.02574682e-01 -3.60543064e-01
-5.26593931e-01 1.06151743e-01 -2.40776152e-02 -4.14707486e-02
-4.51295781e-02 -7.30494592e-02 -3.17157182e-02 1.39949671e-02
-1.22782432e-03]

[-1.88167954e-01 -4.83644733e-01 4.34890968e-03 -2.57322587e-02
-1.51447401e-01 -2.63914644e-02 -6.15592342e-03 7.96858177e-02
3.27382668e-02 2.67389078e-02 9.20382870e-02 -3.18392611e-01
2.68578604e-01 -1.23275722e-01 3.17328549e-02 5.72632653e-03
3.39144863e-02 -8.14525867e-03 1.09752784e-02 -1.15362716e-02
7.06049646e-01]

[2.04173993e-01 -3.96367621e-02 -3.87975848e-01 2.91269514e-01
-1.05550321e-01 9.73045281e-02 3.20498754e-02 9.76391288e-03
1.55564584e-01 -1.99416939e-01 -4.28913985e-01 -2.25860548e-01
-1.03687541e-01 -6.47093887e-02 -4.31658028e-02 6.15331746e-01
-4.92467415e-02 -4.67022775e-02 -2.36926087e-02 9.57774019e-03
2.28627185e-03]

[1.35700316e-01 1.90557726e-02 1.36520644e-01 9.06545647e-02
-3.63364972e-01 -2.81498984e-01 -7.55139294e-01 4.89508855e-02
6.20069087e-02 -3.06118175e-01 5.83890292e-02 1.64019045e-01
1.33786002e-01 1.22313294e-01 -5.91639582e-02 3.41988727e-02
-5.03321190e-02 -1.16030782e-02 4.82283183e-02 8.39872251e-03
6.97025097e-04]

[2.08598028e-01 -3.74368796e-02 -4.21395581e-01 3.06779616e-01
-1.12487097e-01 5.95576813e-02 6.64254024e-03 6.33805069e-02
1.12681024e-01 -6.54487007e-02 -2.30405641e-01 -7.00586149e-02
1.72665696e-02 -6.22325263e-03 1.50573517e-02 -7.54999274e-01
-1.03169660e-02 -1.29741371e-01 2.21429082e-02 -9.90579143e-03
6.70798122e-03]

[-1.47996943e-01 1.34303457e-01 2.14971080e-01 4.61969771e-01
-2.45930881e-01 -2.08905359e-01 3.19681749e-01 -2.69557041e-02
4.67501193e-02 -2.83565698e-01 1.30098707e-01 1.76051060e-01
-3.92993778e-02 -4.97069371e-01 2.20906713e-01 -1.95959734e-02
4.96631431e-02 2.50109751e-01 -1.74625955e-03 -5.39025627e-03
3.16979025e-03]

[2.22203984e-01 -1.75088765e-01 2.82975956e-01 2.75506349e-01
4.15131855e-01 5.83408704e-02 -1.26615385e-02 2.41031213e-01
1.34021259e-01 -2.34198652e-02 2.37596733e-02 -1.50773569e-02
-1.04834281e-03 5.39491776e-02 4.33033572e-02 2.38964936e-02
-3.92175849e-02 3.23077398e-03 7.09194737e-01 2.84287864e-02
-4.69017718e-03]

[-8.14147814e-02 -3.86607442e-01 -5.29263837e-02 -5.31839013e-02
-1.45398554e-01 -1.23097722e-01 9.58612844e-02 -4.91285176e-02
5.77182275e-01 2.87997407e-01 -7.80774284e-02 5.48983973e-01
-2.33206830e-01 8.69925918e-02 -4.85778608e-02 4.23088420e-02
-2.00746351e-02 -1.11847374e-02 -6.51503097e-03 7.60963979e-03
9.32778810e-03]

[-2.83645007e-01 -1.71183225e-01 -1.55460332e-01 2.21880775e-01
2.41640497e-01 -3.12912067e-02 -1.51682073e-01 -3.35539528e-01
-1.12415262e-01 -1.44096328e-01 1.35396750e-01 5.11856821e-04
-2.37681000e-01 1.25940039e-01 2.22870627e-02 -6.36857375e-05
-1.95625116e-02 -1.39394921e-02 2.27704022e-02 -7.04408456e-01
-3.03431175e-03]

[-2.82920925e-01 -1.73513639e-01 -1.55418987e-01 2.25104876e-01

```

2.36917694e-01 -3.27964548e-02 -1.37022230e-01 -3.52814562e-01
-1.20647774e-01 -1.39528455e-01 1.27032685e-01 -6.93025309e-03
-2.06713912e-01 1.35496972e-01 2.71128668e-02 -2.22289761e-02
-1.10341369e-02 1.76812324e-02 -2.48856094e-02 7.07751519e-01
8.25201925e-03]
[ 2.91903776e-01 -1.66361124e-01 -2.36314742e-02 -1.30087585e-02
5.13184184e-02 -2.04178302e-01 3.00074244e-01 -2.18054055e-01
-1.56483310e-01 -1.62188771e-01 1.00191717e-01 1.54206242e-01
3.05842823e-01 8.53628127e-02 -1.95725343e-01 1.47866458e-02
-6.90459267e-01 -7.86660817e-03 -6.00149654e-02 -9.30743184e-03
5.08262186e-03]
[ 3.07221438e-01 -1.42465756e-01 -4.59144039e-03 1.75385839e-02
-2.81525816e-02 -1.81668592e-01 2.51701805e-01 -2.33438738e-01
-1.65041755e-01 -1.68244849e-01 8.71607380e-02 1.75786976e-01
2.04127562e-01 1.49565187e-01 -7.07448517e-02 7.67824464e-02
6.59364171e-01 -3.58345532e-01 3.41863502e-02 -3.98796551e-03
-2.78100191e-03]
[ 2.42434444e-01 -1.40656475e-01 2.30173446e-01 1.43093423e-01
-1.20768217e-01 9.04021137e-02 -1.52818516e-01 -2.46762769e-01
-2.19080847e-01 3.42230618e-01 -1.44996788e-02 -8.93819646e-02
-3.00772588e-01 -4.29692292e-01 -5.35965324e-01 -4.81320398e-02
1.34259731e-02 -5.23248197e-02 4.86122621e-03 6.48368470e-03
-2.51904924e-03]]

```

In []: ****Contributor:** Milliam – Explained variance calculation and dynamic principal compo

In [25]: *# Step 5: Explained Variance + Dynamic Component Selection*

```

# Calculate total variance
total_variance = np.sum(
    sorted_eigenvalues
)

# Calculate explained variance ratio
explained_variance_ratio = (
    sorted_eigenvalues / total_variance
)

# Calculate cumulative explained variance
cumulative_explained_variance = np.cumsum(
    explained_variance_ratio
)

# Select enough components to retain at least 95% variance
num_components = np.argmax(
    cumulative_explained_variance >= 0.95
) + 1

print("Explained variance ratio:")
print(explained_variance_ratio)

print("\nExplained variance percentage:")
print(explained_variance_ratio * 100)

print("\nCummulative explained variance percentage:")
print(cumulative_explained_variance * 100)

```

```
print("\nNumber of selected principal components:")
print(num_components)
```

Explained variance ratio:

```
[3.11082960e-01 1.29820325e-01 8.82484516e-02 6.76483432e-02
 6.01300593e-02 4.97443437e-02 4.07009870e-02 3.87232757e-02
 3.63977920e-02 3.00910586e-02 2.68036366e-02 2.36226894e-02
 2.01209609e-02 1.98441653e-02 1.77852583e-02 1.49239042e-02
 9.39114136e-03 6.98508555e-03 4.93704745e-03 2.86955446e-03
 1.28961099e-04]
```

Explained variance percentage:

```
[3.11082960e+01 1.29820325e+01 8.82484516e+00 6.76483432e+00
 6.01300593e+00 4.97443437e+00 4.07009870e+00 3.87232757e+00
 3.63977920e+00 3.00910586e+00 2.68036366e+00 2.36226894e+00
 2.01209609e+00 1.98441653e+00 1.77852583e+00 1.49239042e+00
 9.39114136e-01 6.98508555e-01 4.93704745e-01 2.86955446e-01
 1.28961099e-02]
```

Cumulative explained variance percentage:

```
[ 31.10829596  44.09032843  52.91517359  59.68000791  65.69301384
  70.66744821  74.73754691  78.60987449  82.24965368  85.25875954
  87.93912321  90.30139214  92.31348823  94.29790476  96.07643059
  97.56882101  98.50793514  99.2064437   99.70014844  99.98710389
 100.          ]
```

Number of selected principal components:

15

Step 6: Project Data onto Principal Components

Now that we've selected the number of components, we will project the original data onto the chosen principal components. Fill in the code to perform the projection.

In [26]: *# Step 6: Project Data onto Principal Components*

```
reduced_data = np.dot(
    standardized_data,
    sorted_eigenvectors[:, :num_components]
)

print("Number of selected components:")
print(num_components)

print("\nOriginal data shape:")
print(standardized_data.shape)

print("\nReduced data shape:")
print(reduced_data.shape)

print("\nFirst 5 rows of reduced data:")
print(reduced_data[:5])
```

Number of selected components:
15

Original data shape:
(2938, 21)

Reduced data shape:
(2938, 15)

First 5 rows of reduced data:

```
[[ -3.84034644 -0.55569827  0.29849804 -0.30153806  1.89410876 -2.23748919
   -1.97630607 -1.24351126 -0.89318586  0.11856165  1.17598349  0.75714253
   -0.57963108  0.85218765 -0.30649352]
 [ -3.63241847 -0.3133572  -0.3190179   0.46246601  1.69821547 -1.85304798
   -1.95131912 -1.26337323 -0.82061592 -0.53080823  0.27574449 -0.13670188
   -0.64759548  0.69214392 -0.41505403]
 [ -3.68525363 -0.56156165 -0.46919913  0.56425814  1.56154344 -1.69745851
   -1.90915978 -1.33687171 -0.41960923 -0.41292077  0.21946135  0.16390177
   -0.76456267  0.74503033 -0.45206507]
 [ -3.66169446 -0.44086963 -0.56806465  0.80148045  1.45896704 -1.42081187
   -2.07937505 -1.22452574 -0.74644172 -0.71240101  0.22266882 -0.07709148
   -0.66276318  0.78156945 -0.49023523]
 [ -3.81168634 -0.43460793 -0.65047309  0.847303   1.47784719 -1.14451102
   -1.91134519 -1.28501448 -0.73317701 -0.66463716  0.22905154 -0.14228792
   -0.69670431  0.77481508 -0.50078554]]
```

Step 7: Output the Reduced Data

Finally, display the reduced data obtained by projecting the original dataset onto the selected principal components.

Contributor: Milliam — Information-loss analysis and interpretation

```
In [29]: # Step 7: Analyze Information Lost Through Dimensionality Reduction

# Calculate variance retained
retained_variance = np.sum(
    explained_variance_ratio[:num_components]
)

# Calculate information lost
information_lost = 1 - retained_variance

# Convert to percentages
retained_percentage = retained_variance * 100
lost_percentage = information_lost * 100

print("Information Retained:")
print(f"{retained_percentage:.2f}%")

print("\nInformation Lost:")
print(f"{lost_percentage:.2f}%")

print("\nNumber of original features:")
```

```

print(standardized_data.shape[1])

print("\nNumber of selected principal components:")
print(num_components)

print("\nAnalysis:")
print(
    f"The selected {num_components} principal component(s) "
    f"retain {retained_percentage:.2f}% of the total variance "
    f"in the original dataset."
)

print(
    f"Approximately {lost_percentage:.2f}% "
    f"of the variance is lost during dimensionality reduction."
)

```

Information Retained:
96.08%

Information Lost:
3.92%

Number of original features:
21

Number of selected principal components:
15

Analysis:
The selected 15 principal component(s) retain 96.08% of the total variance in the original dataset.
Approximately 3.92% of the variance is lost during dimensionality reduction.

Step 8: Visualize Before and After PCA

Now, let's plot the original data and the data after PCA to compare the reduction in dimensions visually.

Contributor: Milliam — Before and after PCA visualization

In [28]: *# Step 8: Visualize Before and After PCA*

```

import matplotlib.pyplot as plt

plt.figure(figsize=(12, 5))

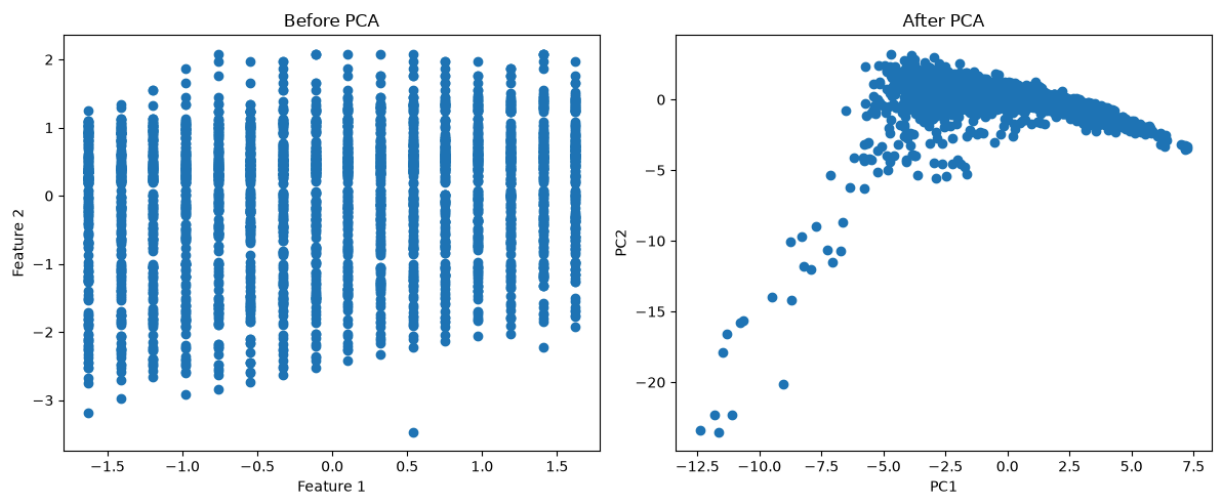
# Original standardized data
plt.subplot(1, 2, 1)
plt.scatter(
    standardized_data[:, 0],
    standardized_data[:, 1]
)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")

```

```
plt.title("Before PCA")

# Reduced PCA data
plt.subplot(1, 2, 2)
plt.scatter(
    reduced_data[:, 0],
    reduced_data[:, 1]
)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("After PCA")

plt.tight_layout()
plt.show()
```



Please make sure you do not use more than 5 lines to answer each of the following points:

1. Interpret the Visual you just created of the before and after PCA: Before PCA, the data is represented using the first two original features. After PCA, the data is represented using PC1 and PC2. The points remain the same observations, but their coordinates are transformed. PC1 captures the largest direction of variance in the data.
2. Explain why you selected the number of principle components, more especially explain what tradeoffs you are making: The number of components was selected dynamically using a 95% cumulative explained variance threshold. Keeping fewer components reduces the number of dimensions and simplifies the data. However, removing components can cause some variance and information to be lost. The trade-off is between reducing dimensionality and retaining as much information as possible.
3. Clearly mention based on your use case/ dataset ("economic activity", "population pressure"), What information is lost when reducing dimensions?: The original dataset contains separate health, education, population, and economic indicators. After PCA, these indicators are represented through combinations of principal components. Some information contained in the dimensions that were removed is therefore lost. In this analysis, approximately 3.92 of the variance is lost after dimensionality reduction.

